

Learning Notes: Explainable Histopathology Image Classification

These notes walk through **every concept used in this project**, why it matters, how it shows up in the code, and a practical example for each. Read alongside `report.md` (results) and `code.py / src/` (implementation).

1. Digital pathology & whole-slide images (WSIs)

What it is: A pathology slide (a thin slice of tissue on glass, stained with Hematoxylin & Eosin — "H&E") is digitized by a slide scanner into a **gigapixel image**. A single slide can be 100,000×200,000 pixels — far too large to load or feed into a CNN directly.

How WSIs are stored: as a **pyramid** of the same image at multiple resolutions, like a zoomable map (think Google Maps tiles). In this project's files (`.tif`, Philips scanner format), `tiffFile.TiffFile(path).pages` gives you each pyramid level as a separate "page":

```
tf = tiffFile.TiffFile("tumor_091.tif")
tf.pages[0].shape # (53760, 61440, 3) <- full resolution
tf.pages[3].shape # (7168, 7680, 3)   <- ~8x downsampled
tf.pages[7].shape # (512, 512, 3)     <- thumbnail
```

Why this matters here: we never touch level 0 (full resolution — would be gigabytes per slide just to load one page). We pick the pyramid level closest to an 8× downsample (`pick_level()` in `extract_patches.py`), which gives images a few thousand pixels per side — small enough to hold entirely in RAM (~150 MB) and tile into patches.

Practical connection: this same pyramid trick is why you can smoothly zoom into a slide in a pathology viewer (like QuPath or ASAP) without it lagging — the viewer just requests whichever pyramid level matches your current zoom.

2. Patch-based classification & PatchCamelyon (PCam)

The problem: you can't feed a 100,000×100,000 pixel image into a CNN. The standard workaround in computational pathology is **patch-based classification**: tile the slide into small fixed-size patches (commonly 96×96 pixels), classify each patch independently, and (for a full clinical pipeline) later aggregate patch predictions into a slide-level diagnosis or heatmap.

PCam is the standard academic benchmark for this: 327,680 labeled 96×96 patches, extracted from CAMELYON16/17 WSIs. This project doesn't download PCam — it **re-derives a small PCam-style dataset from the raw WSIs**, which is instructive because it exposes every design decision PCam's original authors also had to make:

- **How big should a patch be?** 96×96 here (PCam's standard) — big enough to contain meaningful tissue structure, small enough to tile efficiently and keep the CNN lightweight.
- **How do you label a patch from a *pixel-level* tumor mask?** See §4.

Practical example: if you've used Kaggle's "Histopathologic Cancer Detection" competition, that dataset is PCam (slightly modified). Now you know exactly how those 96×96 PNGs were produced from raw slides.

3. Tissue detection (telling tissue apart from empty glass)

A WSI has huge empty regions (bare glass around the tissue sample). Feeding the CNN blank patches wastes data and can teach it to associate "blank" with whichever class happens to be more common.

Technique used: convert RGB → **HSV** color space and threshold on the **Saturation** channel:

```
hsv = cv2.cvtColor(patch_rgb, cv2.COLOR_RGB2HSV)
sat = hsv[:, :, 1].astype(np.float32) / 255.0
is_tissue = (sat > 0.08).mean() >= 0.35    # ≥35% of pixels look "colorful"
```

Why HSV and not RGB? Glass/background is near-white — high in all three RGB channels equally, but critically **low in saturation** (saturation measures "how far from gray/white" a color is). Stained tissue, however light, always has *some* color cast (pink/purple from H&E dyes), so it reads as higher-saturation. This one-line heuristic is a cheap, effective substitute for a full tissue-segmentation network.

Practical example: this is the same principle used by "green screen" chroma-keying in video — HSV separates *how colorful* something is from *how bright* it is, which RGB thresholds can't do cleanly.

4. Labeling patches from a pixel-level mask (the PCam convention)

The tumor annotation is a **pixel mask** (1 = tumor, 0 = normal), not a patch label. A patch that straddles a tumor boundary is ambiguous — is it "tumor" or "normal"? PCam's convention, followed here:

```
center = mask_patch[32:64, 32:64]    # central 32x32 region of the 96x96 patch
if center.any():                      # any tumor pixel in the very center
    label = "tumor"
elif mask_patch.any():                # tumor pixels exist, but only near an edge -> discard
    label = None
else:                                 # zero tumor pixels anywhere in the patch
    label = "normal"
```

Why the center, not the whole patch? Using "any tumor pixel anywhere in the patch" would label huge numbers of mostly-normal, edge-touching patches as "tumor" — teaching the model that a tiny sliver of tumor in a corner looks like the whole class, which hurts label quality. Requiring the tumor signal to be in the *center* keeps the label tied to what's actually visible in the middle of the image (which is also, not coincidentally, where a CNN's receptive field concentrates for a small image).

Practical connection: this exact ambiguous-boundary problem is why medical image segmentation (labeling every pixel) is often preferred over patch classification for final clinical use — but patch classification remains the practical starting point because it's far cheaper to train and label.

5. Stain normalization (Reinhard color transfer)

The problem this project ran into (a real one, not hypothetical): the first training run got **validation AUC 0.97 but held-out test AUC 0.60** — on slides the model had never seen. This is the classic **batch effect**: different slides (different staining batches, scanners, even different days) have systematically different color statistics. A CNN will happily learn "this shade of purple = tumor" if that shortcut works on the slides it was trained on — and that shortcut evaporates on a new slide with slightly different staining.

Fix used: Reinhard color normalization. Convert to **LAB** color space (L = lightness, A/B = color-opponent axes — chosen because it separates *brightness* from *color* better than RGB, and matches human color perception better), then for each patch:

```
lab = cv2.cvtColor(patch, cv2.COLOR_RGB2LAB).astype(np.float32)
mean, std = lab.mean(axis=(0,1)), lab.std(axis=(0,1))
lab_normalized = (lab - mean) / std * REFERENCE_STD + REFERENCE_MEAN
```

This subtracts the patch's *own* mean/std (removing its global color bias) and rescales onto one fixed reference distribution (shared by every patch, every slide). Crucially, it does **not** flatten the patch to a single color — it preserves *relative* internal contrast (a patch with densely-packed dark nuclei still looks different from a sparse one after normalization), because that relative structure is the real diagnostic signal; only the *global, slide-level* stain intensity — the nuisance variable — gets remapped.

Result: after adding this + more training slides, held-out generalization moved (across pipeline iterations) from **AUC 0.60** → **0.71** → **then, with a properly slide-disjoint holdout and more data, ~0.65**, while the in-distribution test AUC reached **0.97**. The two numbers are reported side-by-side in `report.md` specifically because a single number would have hidden this — see §11 for the "why two numbers" discussion.

Practical connection: the same idea underlies Instagram-style photo filters and white-balance correction in cameras — remap an image's color statistics onto a target look while keeping its internal structure intact.

6. Data splitting: slide-level leakage and why it's easy to get wrong

The subtle bug this project hit: the *first* version of this pipeline split patches into train/val *randomly*, without regard to which slide each patch came from. Because patches from the same slide share staining/texture, random validation patches "leaked" information about slide-specific appearance that the model had already partially memorized from training patches of the *same slide*. Result: validation AUC (0.97+) looked great but was measuring "can you recognize patches from a slide you've partly seen," not "can you recognize tumor in new tissue."

The fix: keep an entirely separate, **slide-disjoint holdout set** — 2 whole slides (tumor_084, normal_150) that contribute *zero* patches to train/val/test. Evaluating on this is strictly harder and more honest.

General lesson (not just for histopathology): whenever your data has a natural grouping variable that correlates with appearance/label (patient ID, slide ID, recording session, sensor ID, even "which annotator labeled this"), a naive random train/test split can leak information through that grouping variable and inflate your reported metric. Always ask: "*could the model be learning to recognize the group, instead of the concept I actually care about?*" Scikit-learn's `GroupKFold` / `StratifiedGroupKFold` exist specifically for this.

7. CNN architecture choices

```
Conv(3→32)-BN-ReLU ×2 → MaxPool    # blocks 1-3: standard "VGG-style" feature
Conv(32→64)-BN-ReLU ×2 → MaxPool    # extraction — each block doubles channels,
Conv(64→128)-BN-ReLU ×2 → MaxPool   # halves spatial resolution
Conv(128→256)-BN-ReLU ×2            # block 4: NO pooling (kept for Grad-CAM, see §12)
GlobalAveragePool → Dropout(0.3) → Linear(256→1)
```

- **Two convs per block before pooling** (not one): gives the network more nonlinear capacity per resolution level before throwing away spatial information, a well-established pattern from VGG-style networks.
- **BatchNorm after every conv**: stabilizes training by normalizing activations, letting a higher learning rate be used and typically improving generalization slightly.
- **Global Average Pool (GAP) instead of flattening + big FC layers**: drastically fewer parameters (avoids a huge Flatten→Linear that would dominate parameter count and overfit on a small dataset), and — as a bonus — GAP is exactly the ingredient Grad-CAM's math assumes (see §12).
- **Trained from scratch, no pretrained ImageNet weights**: deliberate choice. Transfer learning usually helps, but (a) it requires downloading pretrained weights, an internet dependency this project's sandboxed environment could not guarantee, and (b) ImageNet's natural-image statistics (photos of dogs, cars, furniture) are a questionable prior for H&E-stained tissue textures anyway. With ~1,800 training patches, a small custom CNN is an appropriately-sized model for the data available — a much deeper network would almost certainly overfit harder.

8. Loss function: `BCEWithLogitsLoss`

Binary classification with one output unit uses **binary cross-entropy on raw logits**, not a 2-unit softmax + `CrossEntropyLoss`. `BCEWithLogitsLoss` combines a sigmoid and the BCE formula in one numerically-stable operation (`torch.sigmoid` followed by `log` can overflow/underflow near 0 or 1 if done as two separate steps — the combined version avoids this). The model outputs a single **logit**; `torch.sigmoid(logit)` turns it into a probability only at evaluation/inference time.

9. Optimizer & learning-rate schedule

- **Adam** (`lr=1e-3`, `weight_decay=1e-4`): adapts the learning rate per-parameter using running estimates of gradient mean/variance — a robust default for small-to-medium CNNs, much less sensitive to the initial learning rate than plain SGD.
- **weight_decay** (L2 regularization): penalizes large weights, a standard defense against overfitting — important here given the small training set.
- **ReduceLROnPlateau** (`mode="max"`, `factor=0.5`, `patience=2`): watches validation AUC; if it hasn't improved for 2 epochs, halves the learning rate. This lets training start fast and automatically "settle" as it approaches a good solution, without manually hand-tuning a decay schedule.

10. Data augmentation for medical images

```
T.RandomHorizontalFlip(0.5), T.RandomVerticalFlip(0.5), T.RandomRotation(90)
T.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.05)
```

Why flips and *arbitrary* rotation (not just 90°/180°/270°) matter more here than for, say, photos of cats: a photo of a cat has a canonical "up" — flipping it upside-down would look wrong. A histopathology patch has **no canonical orientation** — the tissue was sliced and mounted with essentially arbitrary rotation, so flip/rotate augmentation is not just tolerable here, it reflects a genuine real-world invariance the model should have.

Why color jitter is used *in addition to Reinhard normalization* (§5): normalization removes the *systematic, slide-level* color difference; jitter adds *random, per-sample* color noise during training so the model doesn't over-rely on any single color value even within the normalized distribution — the two techniques address different sources of color variation and are complementary, not redundant.

11. Evaluation metrics — and why we report two sets of numbers

- **ROC-AUC** (Receiver Operating Characteristic, Area Under Curve): probability that the model ranks a random positive above a random negative. **Threshold-independent** — this is why it's the primary metric reported: it summarizes ranking quality without committing to any particular decision boundary.
 - **Precision** = $TP / (TP + FP)$: "of everything I called tumor, how much really was?" **Recall** = $TP / (TP + FN)$: "of everything that really was tumor, how much did I catch?" These trade off against each other — in a cancer-screening context, recall (not missing real tumors) is usually weighted more heavily than precision, even at the cost of more false positives sent for a second look.
 - **F1** = harmonic mean of precision and recall — a single number that penalizes ignoring either one.
 - **Confusion matrix**: the raw counts behind all of the above; always worth inspecting directly rather than trusting a single summary number.
 - **Choosing the decision threshold**: a probability output needs a cutoff to become a hard tumor/normal decision. The default 0.5 is arbitrary; this project instead sweeps thresholds on the **validation set** and picks the one maximizing F1, then applies that *same* threshold, unchanged, to both test sets. Tuning the threshold on validation (never on test) avoids quietly leaking test-set information into what looks like a "fixed" decision rule.
 - **Why report test AND holdout_cross_slide separately instead of one number**: ROC-AUC of 0.97 (test) is a true, reproducible number — but only for patches from slides the model partially trained on. 0.65 AUC (cross-slide holdout) is a stricter, more realistic estimate of performance on a genuinely new patient. Reporting only the first would be technically correct but misleading about real-world performance; this is the single most important methodological lesson from this project (see §6 for the root cause).
-

12. Grad-CAM: how it works and why this specific architecture supports it

Grad-CAM (Selvaraju et al., "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization", 2017) answers: "which spatial regions of the input most influenced this prediction?"

Mechanism, matching `GradCAM.__call__` in `gradcam.py`:

1. Run a forward pass, keep the activation map A of a chosen convolutional layer (here, the last conv block, shape $[256, 12, 12]$ — 256 channels, 12×12 spatial grid).
2. Backpropagate the predicted class score to get the gradient of that score w.r.t. every element of A .
3. **Global-average-pool** each channel's gradient into one importance weight per channel: $w_k = \text{mean}(\partial \text{score} / \partial A_k)$ — "how much does turning up channel k overall increase the score?"
4. Combine: $\text{heatmap} = \text{ReLU}(\sum_k w_k * A_k)$ — a weighted sum of the activation channels, keeping only the positive (score-increasing) contributions, then upsampled to the input's resolution and overlaid.

Why this project's CNN removes pooling from the last conv block: Grad-CAM's spatial resolution is exactly the target layer's feature-map size. Pooling away resolution too aggressively would leave, say, a 3×3 heatmap — too coarse to localize anything meaningful inside a 96×96 patch. Keeping the last block at 12×12 (by skipping its MaxPool) gives Grad-CAM enough spatial detail to produce a legible heatmap, at the cost of slightly more compute.

What the resulting heatmaps showed in this project (§5 of `report.md`): correct predictions concentrated tightly on dense, dark (hyperchromatic) nuclear clusters — a genuine pathological signal, not spurious background. Even the model's *mistakes* on unseen slides pointed at plausible cellular structures, which is exactly Grad-CAM's practical value: it turns "the model is wrong" into "the model is wrong, but here's what it was looking at," letting you distinguish *reasonable-but-wrong* (a real interpretability finding, discussed in §5 as evidence of the batch-effect problem) from *nonsensically wrong* (which would suggest a pipeline bug instead).

13. Practical engineering lessons from building this end-to-end

- **Finding real data behind a locked-down network:** this sandbox's network policy blocked Kaggle, HuggingFace, Zenodo, and Google Drive — the "obvious" places to get histopathology data — but allowed generic AWS S3 and Google Cloud Storage traffic. Probing for *which specific public buckets* were reachable (`camelyon-dataset.s3.amazonaws.com`, a legitimate AWS Open Data listing) turned an apparent dead end into access to the actual, real, authoritative CAMELYON16 raw data — often more productive than reaching for a pre-packaged, tutorial-friendly mirror.
- **Whole-slide images are pyramidal for a reason:** trying to load a WSI at full resolution would have been infeasible (tens of gigabytes, decoded); working with the pyramid level matching your actual needed resolution is not an optimization, it's a requirement.
- **A model can look great and still be wrong** — the val-AUC-0.97 vs test-AUC-0.60 discovery in this project is a direct, first-hand encounter with why rigorous, leakage-free evaluation matters more than a single impressive-looking number, and why "the model is 97% accurate" always deserves the follow-up question "accurate on what, exactly, and measured how?"

14. Glossary

Term	Meaning here
WSI	Whole-Slide Image — a digitized pathology slide, gigapixel-scale
Patch	A small (96×96) tile cut from a WSI, the unit fed to the CNN
H&E	Hematoxylin & Eosin — the standard pathology tissue stain (purple/pink)
Batch effect	Systematic, non-biological variation between data batches (slides/scanners) that a model can wrongly learn to exploit
Reinhard normalization	Color-transfer technique that remaps an image's color statistics onto a fixed reference
GAP	Global Average Pooling — collapse a $[C, H, W]$ feature map to $[C]$ by averaging over space
Grad-CAM	Gradient-weighted Class Activation Mapping — a post-hoc CNN explainability technique
ROC-AUC	Area under the Receiver Operating Characteristic curve; threshold-independent ranking quality
Slide-disjoint split	A train/test split where no slide contributes patches to both sides